

Pocket Ledger

Personal expense tracker: complete front-end and back-end source code

7 files · 1272 lines · Repository: [mohan196/git-test](#) · Generated 29 Sep 2026

Technology stack

Front end	HTML5, hand-written CSS (custom properties, Grid, Flexbox, media queries) and plain JavaScript. No framework.
Charts	Chart.js 4.4.1, loaded from cdnjs.cloudflare.com : stacked bar, donut, line and bar charts with tooltips.
Fonts	Google Fonts: Bricolage Grotesque (headings), IBM Plex Sans (text), IBM Plex Mono (numbers).
Back end	Node.js 22 + Express 5 REST API (backend/). Validates input, serves the dashboard, CSV export, optional password login.
Database	SQLite via Node's built-in <code>node:sqlite</code> : tables expenses, budgets, settings. Opened as a plain file, the page uses browser <code>localStorage</code> instead.

Features

- Summary cards: spent this month (vs. last month), budget remaining with status, daily average with projection, top category
- Six-month stacked bar chart of spending by category
- Donut chart of the month's category split, with amounts and percentages
- Running total vs. budget pace, compared with the previous month
- Per-category budgets with editable limits and On track / Near limit / Over budget labels
- Spending by day of week
- Add-expense form; transactions table with category filter, search and delete
- Month and currency selectors (INR, USD, EUR, GBP); light and dark themes; phone layout

Expense categories

 Housing	 Food & Dining	 Transport	 Utilities
 Shopping	 Health	 Entertainment	 Other

Code layout

Front end: `index.html`

Lines 1–11	Document head: fonts and Chart.js
Lines 12–196	CSS: theme, layout and components
Lines 199–324	HTML: page structure
Lines 325–744	JavaScript: data, charts and interactions

Back end

<code>backend/package.json</code>	Dependencies and npm scripts · 19 lines
<code>backend/src/categories.js</code>	Category ids, default budgets, currencies · 15 lines
<code>backend/src/db.js</code>	SQLite schema and data access · 98 lines
<code>backend/src/server.js</code>	Express REST API, validation, login · 222 lines
<code>backend/test/api.test.js</code>	Automated API tests · 159 lines
<code>Dockerfile</code>	Container image for deployment · 12 lines

Run: `cd backend, npm install, npm start`, then open `http://localhost:3000`. Or open `index.html` directly to use it without a server.

Document head: fonts and Chart.js

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
6 <!-- ARTIFACT-START -->
7 <title>Pocket Ledger</title>
8 <link rel="preconnect" href="https://fonts.googleapis.com">
9 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10 <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:opsz,wght@12..96,
  500;12..96,700&family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono:wght@400;500&display=swap">
11 <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.1/chart.umd.min.js"></script>
```

CSS: theme, layout and components

```
12 <style>
13 :root {
14   --bg: #f3f5f2;
15   --surface: #fcfcfb;
16   --surface-2: #eefled;
17   --ink: #121715;
18   --ink-2: #4c5652;
19   --muted: #7a8480;
20   --line: #dfe4df;
21   --grid: #e9ece8;
22   --accent: #1d4d45;
23   --accent-ink: #ffffff;
24   --good: #0ca30c; --good-ink: #0a6b0a;
25   --warn: #fab219; --warn-ink: #8a5a00;
26   --bad: #d03b3b; --bad-ink: #b02a2a;
27   --c1: #2a78d6; --c2: #eb6834; --c3: #1baf7a; --c4: #eda100;
28   --c5: #e87ba4; --c6: #008300; --c7: #4a3aa7; --c8: #e34948;
29   --ghost: #b8c0bc;
30   --display: "Bricolage Grotesque", "Segoe UI", system-ui, sans-serif;
31   --sans: "IBM Plex Sans", "Segoe UI", system-ui, sans-serif;
32   --mono: "IBM Plex Mono", ui-monospace, "SFMono-Regular", Menlo, monospace;
33 }
34 @media (prefers-color-scheme: dark) {
35   :root:not([data-theme="light"]) {
36     color-scheme: dark;
37     --bg: #101312; --surface: #1a1d1c; --surface-2: #232826;
38     --ink: #f1f4f2; --ink-2: #bcc5c1; --muted: #8b9591;
39     --line: #2e3432; --grid: #262b29;
40     --accent: #7fcdcd; --accent-ink: #0d1f1b;
41     --good-ink: #5fd35f; --warn-ink: #fab219; --bad-ink: #f07a7a;
42     --c1: #3987e5; --c2: #d95926; --c3: #199e70; --c4: #c98500;
43     --c5: #d55181; --c6: #008300; --c7: #9085e9; --c8: #e66767;
44     --ghost: #4d5552;
45   }
46 }
47 :root[data-theme="dark"] {
48   color-scheme: dark;
49   --bg: #101312; --surface: #1a1d1c; --surface-2: #232826;
50   --ink: #f1f4f2; --ink-2: #bcc5c1; --muted: #8b9591;
51   --line: #2e3432; --grid: #262b29;
52   --accent: #7fcdcd; --accent-ink: #0d1f1b;
53   --good-ink: #5fd35f; --warn-ink: #fab219; --bad-ink: #f07a7a;
54   --c1: #3987e5; --c2: #d95926; --c3: #199e70; --c4: #c98500;
55   --c5: #d55181; --c6: #008300; --c7: #9085e9; --c8: #e66767;
56   --ghost: #4d5552;
57 }
58
59 * { box-sizing: border-box; }
60 [hidden] { display: none !important; }
61 html, body { margin: 0; }
62 body {
63   background: var(--bg);
64   color: var(--ink);
65   font: 400 14px/1.5 var(--sans);
66   padding-inline: 16px;
67   padding-block: 20px 48px;
68 }
69 .wrap { max-width: 1280px; margin: 0 auto; display: grid; gap: 16px; }
70 .num { font-family: var(--mono); font-variant-numeric: tabular-nums; }
71
72 /* Header */
73 header.top { display: flex; flex-wrap: wrap; align-items: end; justify-content: space-between; gap: 16px;
```

```

    padding-block: 4px 8px; }
74 .brand h1 { font: 700 clamp(26px, 4vw, 36px)/1.05 var(--display); margin: 0; letter-spacing: -0.02em; text-align: center; }
75 .brand p { margin: 4px 0 0; color: var(--ink-2); }
76 .controls { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; }
77 label.inline { display: flex; flex-direction: column; gap: 2px; font-size: 11px; text-transform: uppercase; }
78 e; letter-spacing: 0.06em; color: var(--muted); }
79 select, input, button { font: inherit; color: inherit; }
80 select, input {
81     background: var(--surface); border: 1px solid var(--line); border-radius: 8px;
82     padding: 7px 10px; min-height: 36px; color: var(--ink);
83 }
84 select:focus-visible, input:focus-visible, button:focus-visible { outline: 2px solid var(--accent); outline-offset: 2px; }
85 .btn {
86     border: 1px solid var(--line); background: var(--surface); color: var(--ink);
87     border-radius: 8px; padding: 7px 14px; min-height: 36px; cursor: pointer; font-weight: 500;
88 }
89 .btn:hover { background: var(--surface-2); }
90 .btn.primary { background: var(--accent); color: var(--accent-ink); border-color: var(--accent); }
91 .btn.primary:hover { filter: brightness(1.08); }
92 .btn.danger-armed { border-color: var(--bad); color: var(--bad-ink); }
93 .notice {
94     display: flex; flex-wrap: wrap; gap: 8px 16px; align-items: center; justify-content: space-between;
95     background: var(--surface-2); border: 1px dashed var(--line); border-radius: 10px; padding: 10px 14px;
96     color: var(--ink-2);
97 }
98 .notice strong { color: var(--ink); }
99 .notice.error { border: 1px solid var(--bad); color: var(--bad-ink); border-style: solid; }
100 .store { display: inline-flex; align-items: center; gap: 5px; margin-left: 6px; font-size: 12px; color: var(--muted); white-space: nowrap; }
101 .store::before { content: ""; width: 7px; height: 7px; border-radius: 50%; background: var(--muted); }
102 .store.server::before { background: var(--good); }
103 .foot a { color: var(--accent); }
104 /* KPI tiles */
105 .kpis { display: grid; grid-template-columns: repeat(4, minmax(0, 1fr)); gap: 16px; }
106 .kpi { background: var(--surface); border: 1px solid var(--line); border-radius: 12px; padding: 16px 18px; }
107 .kpi .label { font-size: 11px; text-transform: uppercase; letter-spacing: 0.07em; color: var(--muted); font-weight: 600; }
108 .kpi .value { font: 500 clamp(22px, 2.6vw, 30px)/1.1 var(--mono); font-variant-numeric: tabular-nums; letter-spacing: -0.02em; }
109 .kpi .sub { color: var(--ink-2); font-size: 13px; display: flex; gap: 6px; align-items: center; flex-wrap: wrap; }
110 .meter { height: 6px; border-radius: 3px; background: var(--surface-2); overflow: hidden; }
111 .meter > span { display: block; height: 100%; border-radius: 3px; background: var(--accent); }
112 .pill { display: inline-flex; align-items: center; gap: 5px; padding: 1px 8px 1px 6px; border-radius: 999px; font-size: 12px; font-weight: 600; border: 1px solid currentColor; line-height: 1.6; white-space: nowrap; }
113 .pill::before { content: ""; width: 7px; height: 7px; border-radius: 50%; background: var(--dot); }
114 .pill.good { color: var(--good-ink); --dot: var(--good); }
115 .pill.warn { color: var(--warn-ink); --dot: var(--warn); }
116 .pill.bad { color: var(--bad-ink); --dot: var(--bad); }
117 .delta.up { color: var(--bad-ink); } .delta.down { color: var(--good-ink); }
118 /* Panels */
119 .grid { display: grid; grid-template-columns: repeat(12, minmax(0, 1fr)); gap: 16px; }
120 .panel { background: var(--surface); border: 1px solid var(--line); border-radius: 12px; padding: 16px 18px; display: grid; gap: 12px; align-content: start; min-width: 0; }
121 .panel h2 { font: 600 16px/1.3 var(--display); margin: 0; letter-spacing: -0.01em; }
122 .panel .head { display: flex; justify-content: space-between; align-items: baseline; gap: 12px; flex-wrap: wrap; }
123 .panel .hint { color: var(--muted); font-size: 12px; }
124 .span-8 { grid-column: span 8; } .span-7 { grid-column: span 7; } .span-5 { grid-column: span 5; } .span-4 { grid-column: span 4; }
125 .stack { display: grid; gap: 16px; align-content: start; min-width: 0; }
126 .chart-box { position: relative; height: 280px; }
127 .chart-box.short { height: 200px; }
128 .chart-box.tall { height: 400px; }
129 .chart-box.mid { height: 340px; }
130 .legend { display: flex; flex-wrap: wrap; gap: 6px 14px; font-size: 12px; color: var(--ink-2); }
131 .legend span { display: inline-flex; align-items: center; gap: 6px; }
132 .swatch { width: 10px; height: 10px; border-radius: 3px; background: var(--c); flex: none; }
133 .swatch.line { height: 2px; border-radius: 1px; width: 14px; }
134 .swatch.dash { height: 0; width: 14px; border-top: 2px dashed var(--c); background: none; }

```

```

138
139 /* Donut */
140 .donut-wrap { position: relative; height: 220px; }
141 .donut-center { position: absolute; inset: 0; display: grid; place-content: center; text-align: center; pointer-events: none; }
142 .donut-center .v { font: 500 20px/1.1 var(--mono); font-variant-numeric: tabular-nums; }
143 .donut-center .l { font-size: 11px; color: var(--muted); text-transform: uppercase; letter-spacing: 0.07em; }
144 .cat-list { display: grid; gap: 2px; margin: 0; padding: 0; list-style: none; }
145 .cat-list li { display: grid; grid-template-columns: auto 1fr auto auto; gap: 10px; align-items: center; padding: 5px 4px; border-radius: 6px; }
146 .cat-list li:hover { background: var(--surface-2); }
147 .cat-list .amt { font-family: var(--mono); font-variant-numeric: tabular-nums; }
148 .cat-list .pct { font-family: var(--mono); font-variant-numeric: tabular-nums; color: var(--muted); width: 3.5em; text-align: right; }
149
150 /* Budgets */
151 .budgets { display: grid; gap: 12px; margin: 0; padding: 0; list-style: none; }
152 .budgets li { display: grid; gap: 5px; }
153 .budgets .row { display: flex; justify-content: space-between; align-items: center; gap: 8px; flex-wrap: wrap; }
154 .budgets .name { display: inline-flex; align-items: center; gap: 8px; font-weight: 500; }
155 .budgets .fig { font-family: var(--mono); font-variant-numeric: tabular-nums; font-size: 13px; color: var(--ink-2); display: inline-flex; align-items: center; gap: 4px; }
156 .budgets input { width: 92px; min-height: 28px; padding: 2px 6px; font-family: var(--mono); font-size: 13px; text-align: right; }
157 .bar { position: relative; height: 8px; border-radius: 4px; background: var(--surface-2); overflow: hidden; }
158 .bar > span { position: absolute; inset: 0 auto 0 0; border-radius: 4px; background: var(--c); }
159 .bar.over > span { background: repeating-linear-gradient(135deg, var(--c) 0 6px, color-mix(in srgb, var(--c) 60%, var(--surface)) 6px 9px); }
160
161 /* Form */
162 form.add { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
163 form.add .full { grid-column: 1 / -1; }
164 form.add label { display: grid; gap: 4px; font-size: 12px; color: var(--ink-2); font-weight: 500; }
165 form.add input, form.add select { width: 100%; }
166 .form-msg { font-size: 13px; min-height: 1.5em; color: var(--ink-2); }
167 .form-msg.err { color: var(--bad-ink); }
168
169 /* Transactions */
170 .filters { display: flex; flex-wrap: wrap; gap: 6px; }
171 .chip { display: inline-flex; align-items: center; gap: 6px; border: 1px solid var(--line); background: var(--surface); color: var(--ink-2); border-radius: 999px; padding: 3px 10px; font-size: 12px; cursor: pointer; min-height: 28px; }
172 .chip[aria-pressed="true"] { background: var(--ink); color: var(--bg); border-color: var(--ink); }
173 .chip .swatch { width: 8px; height: 8px; border-radius: 50%; }
174 .table-scroll { overflow-x: auto; max-height: 460px; overflow-y: auto; border-top: 1px solid var(--line); }
175 table { width: 100%; border-collapse: collapse; font-size: 13px; }
176 th, td { text-align: left; padding: 8px 6px; border-bottom: 1px solid var(--grid); white-space: nowrap; }
177 th { position: sticky; top: 0; background: var(--surface); font-size: 11px; text-transform: uppercase; letter-spacing: 0.06em; color: var(--muted); font-weight: 600; }
178 td.amt, th.amt { text-align: right; font-family: var(--mono); font-variant-numeric: tabular-nums; }
179 td.note { white-space: normal; min-width: 140px; color: var(--ink-2); }
180 .cat-tag { display: inline-flex; align-items: center; gap: 6px; }
181 .del { border: 0; background: none; color: var(--muted); cursor: pointer; font-size: 16px; line-height: 1; padding: 4px 8px; border-radius: 6px; }
182 .del:hover { color: var(--bad-ink); background: var(--surface-2); }
183 .empty { color: var(--muted); padding: 16px 4px; text-align: center; }
184 .foot { display: flex; justify-content: space-between; gap: 8px; flex-wrap: wrap; color: var(--ink-2); font-size: 13px; }
185
186 @media (max-width: 1000px) {
187   .span-8, .span-7, .span-5, .span-4 { grid-column: 1 / -1; }
188   .kpis { grid-template-columns: repeat(2, minmax(0, 1fr)); }
189 }
190 @media (max-width: 460px) {
191   .kpis { grid-template-columns: 1fr; }
192   form.add { grid-template-columns: 1fr; }
193   .chart-box, .chart-box.tall, .chart-box.mid { height: 260px; }
194 }
195 @media (prefers-reduced-motion: reduce) { * { transition: none !important; animation: none !important; } }
196 </style>
197 <!-- ARTIFACT-HEAD-END -->
198 </head>

```

HTML: page structure

```
199 <body>
200 <!-- ARTIFACT-BODY-START -->
201 <div class="wrap">
202   <header class="top">
203     <div class="brand">
204       <h1>Pocket Ledger</h1>
205       <p>Where your money went, by month and by category. <span class="store" id="storeLabel">Saved in this
  browser</span></p>
206     </div>
207     <div class="controls">
208       <label class="inline" for="monthSel">Month
209       <select id="monthSel"></select>
210     </label>
211       <label class="inline" for="curSel">Currency
212       <select id="curSel">
213         <option value="INR">₹ INR</option>
214         <option value="USD">$ USD</option>
215         <option value="EUR">€ EUR</option>
216         <option value="GBP">£ GBP</option>
217       </select>
218     </label>
219   </div>
220 </header>
221
222   <div class="notice error" id="syncMsg" role="alert" hidden></div>
223
224   <div class="notice" id="sampleNotice" hidden>
225     <span><strong>You're looking at sample data.</strong> Add your own expenses below, or clear the sample
  to start fresh. <span id="storeText">Everything is saved in this browser only.</span></span>
226     <button class="btn" id="clearBtn" type="button">Clear sample data</button>
227   </div>
228
229   <section class="kpis" aria-label="Month summary">
230     <div class="kpi">
231       <span class="label">Spent this month</span>
232       <span class="value" id="kSpent"></span>
233       <span class="sub" id="kSpentSub"></span>
234     </div>
235     <div class="kpi">
236       <span class="label">Budget remaining</span>
237       <span class="value" id="kLeft"></span>
238       <div class="meter" aria-hidden="true"><span id="kMeter" style="width:0%"></span></div>
239       <span class="sub" id="kLeftSub"></span>
240     </div>
241     <div class="kpi">
242       <span class="label">Daily average</span>
243       <span class="value" id="kDaily"></span>
244       <span class="sub" id="kDailySub"></span>
245     </div>
246     <div class="kpi">
247       <span class="label">Top category</span>
248       <span class="value" id="kTop" style="font-family:var(--display);font-weight:600;letter-spacing:-0.01e
  m"></span>
249       <span class="sub" id="kTopSub"></span>
250     </div>
251   </section>
252
253   <section class="grid">
254     <div class="panel span-8">
255       <div class="head"><h2>Six-month spending by category</h2><span class="hint">Hover a bar for the break
  down</span></div>
256       <div class="legend" id="catLegend"></div>
257       <div class="chart-box tall"><canvas id="trendChart" aria-label="Stacked bar chart of monthly spending
  by category"></canvas></div>
258     </div>
259     <div class="panel span-4">
260       <div class="head"><h2>Where it went</h2><span class="hint" id="donutHint"></span></div>
261       <div class="donut-wrap">
262         <canvas id="donutChart" aria-label="Donut chart of spending share by category"></canvas>
263         <div class="donut-center"><span class="v num" id="donutTotal"></span><span class="l">total</span></div>
  </div>
264     </div>
265     <ul class="cat-list" id="catList"></ul>
266   </div>
267
268   <div class="panel span-7">
269     <div class="head"><h2>Running total vs. budget pace</h2><span class="hint">Cumulative spend through t
```

```

    he month</span></div>
270     <div class="legend">
271       <span><i class="swatch line" style="--c:var(--c1)"></i>This month</span>
272       <span><i class="swatch line" style="--c:var(--ghost)"></i>Previous month</span>
273       <span><i class="swatch dash" style="--c:var(--muted)"></i>Budget pace</span>
274     </div>
275     <div class="chart-box mid"><canvas id="paceChart" aria-label="Line chart of cumulative spending again
    st budget pace"></canvas></div>
276   </div>
277   <div class="panel span-5">
278     <div class="head"><h2>Category budgets</h2><span class="hint">Edit a limit to change it</span></div>
279     <ul class="budgets" id="budgetList"></ul>
280   </div>
281
282   <div class="stack span-4">
283     <div class="panel">
284       <h2>Add an expense</h2>
285       <form class="add" id="addForm" novalidate>
286         <label for="fAmount">Amount
287         <input id="fAmount" type="number" inputmode="decimal" min="0" step="0.01" placeholder="0.00" re
    quired>
288         </label>
289         <label for="fDate">Date
290         <input id="fDate" type="date" required>
291         </label>
292         <label class="full" for="fCat">Category
293         <select id="fCat"></select>
294         </label>
295         <label class="full" for="fNote">Note
296         <input id="fNote" type="text" maxlength="80" placeholder="e.g. Weekly groceries">
297         </label>
298         <button class="btn primary full" type="submit">Add expense</button>
299         <div class="form-msg full" id="formMsg" role="status"></div>
300       </form>
301     </div>
302     <div class="panel">
303       <div class="head"><h2>By day of week</h2><span class="hint">Selected month</span></div>
304       <div class="chart-box short"><canvas id="dowChart" aria-label="Bar chart of spending by day of week
    "></canvas></div>
305     </div>
306   </div>
307
308   <div class="panel span-8">
309     <div class="head">
310       <h2>Transactions</h2>
311       <input id="search" type="search" placeholder="Search notes" aria-label="Search notes" style="min-he
    ight:32px;padding:4px 10px">
312     </div>
313     <div class="filters" id="filters" role="group" aria-label="Filter by category"></div>
314     <div class="table-scroll">
315       <table>
316         <thead><tr><th>Date</th><th>Category</th><th>Note</th><th class="amt">Amount</th><th><span class=
    "sr" hidden>Actions</span></th></tr></thead>
317         <tbody id="txBody"></tbody>
318       </table>
319     </div>
320     <div class="foot"><span id="txCount"></span><a id="csvLink" href="api/export.csv" hidden>Download all
    as CSV</a><span id="txSum" class="num"></span></div>
321   </div>
322 </section>
323 </div>
324

```

JavaScript: data, charts and interactions

```

325 <script>
326 (() => {
327   const CATS = [
328     { id: "housing", name: "Housing", v: "--c1", budget: 18000 },
329     { id: "food", name: "Food & Dining", v: "--c2", budget: 12000 },
330     { id: "transport", name: "Transport", v: "--c3", budget: 6000 },
331     { id: "utilities", name: "Utilities", v: "--c4", budget: 4500 },
332     { id: "shopping", name: "Shopping", v: "--c5", budget: 6000 },
333     { id: "health", name: "Health", v: "--c6", budget: 3000 },
334     { id: "fun", name: "Entertainment", v: "--c7", budget: 3500 },
335     { id: "other", name: "Other", v: "--c8", budget: 2500 },
336   ];
337   const CAT = Object.fromEntries(CATS.map(c => [c.id, c]));
338   const KEY = "pocket-ledger-v1";

```

```

339 const LOCALES = { INR: "en-IN", USD: "en-US", EUR: "en-IE", GBP: "en-GB" };
340
341 const pad = n => String(n).padStart(2, "0");
342 const ymd = d => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
343 const ym = d => `${d.getFullYear()}-${pad(d.getMonth() + 1)}`;
344 const today = new Date();
345 const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 8).padEnd(6, "0");
346
347 // ----- Sample data (deterministic) -----
348 function mulberry32(a) { return () => { a |= 0; a = a + 0x6D2B79F5 | 0; let t = Math.imul(a ^ a >>> 15, 1
  | a); t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t; return ((t ^ t >>> 14) >>> 0) / 4294967296; }; }
349 function buildSample() {
350   const r = mulberry32(42);
351   const rand = (lo, hi) => Math.round((lo + r() * (hi - lo)) / 10) * 10;
352   const out = [];
353   for (let m = 5; m >= 0; m--) {
354     const first = new Date(today.getFullYear(), today.getMonth() - m, 1);
355     const dim = new Date(first.getFullYear(), first.getMonth() + 1, 0).getDate();
356     const lastDay = m === 0 ? today.getDate() : dim;
357     const add = (day, cat, amount, note) => {
358       if (day > lastDay) return;
359       out.push({ id: uid(), date: ymd(new Date(first.getFullYear(), first.getMonth(), day)), cat, amount,
  note });
360     };
361     const any = () => 1 + Math.floor(r() * dim);
362     add(1, "housing", 18000, "Rent");
363     add(5, "utilities", rand(1600, 2700), "Electricity bill");
364     add(10, "utilities", 799, "Home internet");
365     add(12, "utilities", 399, "Mobile recharge");
366     for (let i = 0; i < 5; i++) add(any(), "food", rand(700, 2300), ["Groceries", "Vegetables & fruit", "
  Supermarket run", "Milk & bakery"][i % 4]);
367     for (let i = 0, n = 5 + Math.floor(r() * 5); i < n; i++) add(any(), "food", rand(220, 1300), ["Dinner
  out", "Lunch with team", "Food delivery", "Coffee & snacks", "Weekend brunch"][i % 5]);
368     for (let i = 0, n = 8 + Math.floor(r() * 6); i < n; i++) add(any(), "transport", rand(80, 420), ["Cab
  ride", "Auto", "Metro card top-up", "Parking"][i % 4]);
369     add(any(), "transport", rand(1500, 2400), "Fuel");
370     if (r() > 0.4) add(any(), "transport", rand(1500, 2400), "Fuel");
371     for (let i = 0, n = 2 + Math.floor(r() * 4); i < n; i++) add(any(), "shopping", rand(450, 3400), ["Cl
  othes", "Household items", "Online order", "Shoes", "Gift"][i % 5]);
372     for (let i = 0, n = 1 + Math.floor(r() * 2); i < n; i++) add(any(), "health", rand(300, 2400), ["Phar
  macy", "Doctor visit", "Gym membership"][i % 3]);
373     add(3, "fun", 649, "Streaming subscription");
374     for (let i = 0, n = 1 + Math.floor(r() * 4); i < n; i++) add(any(), "fun", rand(250, 1600), ["Movie t
  ickets", "Concert", "Games", "Books"][i % 4]);
375     for (let i = 0, n = 1 + Math.floor(r() * 3); i < n; i++) add(any(), "other", rand(200, 1400), ["Hairc
  ut", "Donation", "Stationery", "Laundry"][i % 4]);
376   }
377   return out;
378 }
379
380 // ----- State -----
381 // Data lives on the backend (backend/ folder) when the page is served by it.
382 // Opened any other way (as a file, on GitHub Pages), it falls back to this browser's localStorage.
383 const DEFAULT_BUDGETS = Object.fromEntries(CATS.map(c => [c.id, c.budget]));
384 const sampleLedger = () => ({ expenses: buildSample(), budgets: { ...DEFAULT_BUDGETS }, currency: "INR",
  sample: true });
385 function readLocal() {
386   try { const s = JSON.parse(localStorage.getItem(KEY)); if (s && Array.isArray(s.expenses)) return s; }
  catch (e) {}
387   return null;
388 }
389 function saveLocal() { try { localStorage.setItem(KEY, JSON.stringify(state)); } catch (e) {} }
390 const state = readLocal() || sampleLedger();
391 state.budgets = { ...DEFAULT_BUDGETS, ...state.budgets };
392 // Categories that no longer exist (the retired SIP and EMI): sample rows are regenerated, the user's own
  rows move to Other.
393 if (state.sample && state.expenses.some(e => !CAT[e.cat])) { state.expenses = buildSample(); saveLocal(); }
394 else if (state.expenses.some(e => !CAT[e.cat])) { state.expenses.forEach(e => { if (!CAT[e.cat]) e.cat =
  "other"; }); saveLocal(); }
395
396 let serverMode = false;
397 const toApi = e => ({ id: e.id, date: e.date, category: e.cat, amount: e.amount, note: e.note });
398 const fromApi = e => ({ id: e.id, date: e.date, cat: CAT[e.category] ? e.category : "other", amount: e.am
  ount, note: e.note });
399 async function api(method, path, body) {
400   const res = await fetch("api/" + path, {
401     method, cache: "no-store",
402     headers: body ? { "Content-Type": "application/json" } : {},

```

```

403     body: body ? JSON.stringify(body) : undefined,
404   });
405   if (!res.ok) {
406     let msg = `the server answered ${res.status}`;
407     try { msg = (await res.json()).error || msg; } catch (e) {}
408     throw new Error(msg);
409   }
410   return res.status === 204 ? null : res.json();
411 }
412 async function detectServer() {
413   if (!/^https?:$/.test(location.protocol)) return false;
414   try { const r = await fetch("api/health", { cache: "no-store" }); return r.ok && (await r.json()).ok ==
    = true; }
415   catch (e) { return false; }
416 }
417 async function loadFromServer() {
418   let s = await api("GET", "state");
419   if (!s.initialized) {
420     // First run: move this browser's own ledger to the server if it has one, otherwise start with sample
    data.
421     const local = readLocal();
422     const seed = local && !local.sample ? local : sampleLedger();
423     await api("POST", "import", { expenses: seed.expenses.map(toApi), budgets: { ...DEFAULT_BUDGETS, ...s
    eed.budgets }, currency: seed.currency, sample: !!seed.sample });
424     s = await api("GET", "state");
425   }
426   state.expenses = s.expenses.map(fromApi);
427   state.budgets = { ...DEFAULT_BUDGETS, ...s.budgets };
428   state.currency = s.currency;
429   state.sample = s.sample;
430 }
431 // Saves a change: sends it to the server in server mode, or writes localStorage otherwise.
432 function persist(request) {
433   if (!serverMode) { saveLocal(); return; }
434   request().catch(err => showSyncError(`Couldn't save to the server: ${err.message}. Reload the page to s
    ee what was saved.`));
435 }
436 function showSyncError(text) { $("syncMsg").textContent = text; $("syncMsg").hidden = false; }
437 let month = ym(today);
438 let filterCat = "all";
439 let query = "";
440
441 let fmt, fmtShort;
442 function setCurrency() {
443   const loc = LOCALES[state.currency] || "en-US";
444   fmt = new Intl.NumberFormat(loc, { style: "currency", currency: state.currency, maximumFractionDigits:
    0 });
445   fmtShort = new Intl.NumberFormat("en-US", { style: "currency", currency: state.currency, notation: "com
    pact", maximumFractionDigits: 1 });
446 }
447 const money = v => fmt.format(v);
448
449 // ----- Helpers -----
450 const $ = id => document.getElementById(id);
451 const css = v => getComputedStyle(document.documentElement).getPropertyValue(v).trim();
452 const color = c => css(CAT[c].v);
453 const inMonth = key => state.expenses.filter(e => e.date.startsWith(key));
454 const sum = arr => arr.reduce((a, e) => a + e.amount, 0);
455 const monthDate = key => { const [y, m] = key.split("-").map(Number); return new Date(y, m - 1, 1); };
456 const shiftMonth = (key, n) => { const d = monthDate(key); return ym(new Date(d.getFullYear(), d.getMonth
    () + n, 1)); };
457 const monthLabel = (key, style = "long") => monthDate(key).toLocaleDateString("en-US", { month: style, ye
    ar: "numeric" });
458 const daysIn = key => { const d = monthDate(key); return new Date(d.getFullYear(), d.getMonth() + 1, 0).g
    etDate(); };
459 const byCat = list => { const o = Object.fromEntries(CATS.map(c => [c.id, 0])); list.forEach(e => { o[e.c
    at] = (o[e.cat] || 0) + e.amount; }); return o; };
460 const esc = s => String(s).replace(/&lt;"/g, ch => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&qu
    ot;", '"': "&#39;" }[ch]));
461 const totalBudget = () => CATS.reduce((a, c) => a + (state.budgets[c.id] || 0), 0);
462 const status = pct => pct > 1 ? ["bad", "Over budget"] : pct >= 0.8 ? ["warn", "Near limit"] : ["good", "
    On track"];
463
464 // ----- Controls -----
465 function fillMonths() {
466   const keys = new Set(state.expenses.map(e => e.date.slice(0, 7)));
467   keys.add(ym(today));
468   const sorted = [...keys].sort().reverse();
469   if (!sorted.includes(month)) month = sorted[0];

```

```

470     $("monthSel").innerHTML = sorted.map(k => `<option value="${k}"${k === month ? " selected" : ""}>${mont
    ↪ hLabel(k)}</option>`).join("");
471 }
472 $("monthSel").addEventListener("change", e => { month = e.target.value; render(); });
473 $("curSel").value = state.currency;
474 $("curSel").addEventListener("change", e => {
475     state.currency = e.target.value; setCurrency(); render();
476     persist() => api("PUT", "settings", { currency: state.currency });
477 });
478 $("fCat").innerHTML = CATS.map(c => `<option value="${c.id}">${c.name}</option>`).join("");
479 $("fDate").value = ymd(today);
480 $("catLegend").innerHTML = CATS.map(c => `<span><i class="swatch" style="--c:var(${c.v})"></i>${c.name}</
    ↪ span>`).join("");
481
482 $("addForm").addEventListener("submit", e => {
483     e.preventDefault();
484     const amount = parseFloat($("fAmount").value);
485     const date = $("fDate").value;
486     const msg = $("formMsg");
487     if (!(amount > 0)) { msg.className = "form-msg full err"; msg.textContent = "Enter an amount greater th
    ↪ an zero."; $("fAmount").focus(); return; }
488     if (!/^\\d{4}-\\d{2}-\\d{2}$/.test(date)) { msg.className = "form-msg full err"; msg.textContent = "Pick a
    ↪ date for this expense."; $("fDate").focus(); return; }
489     const cat = $("fCat").value;
490     const note = $("fNote").value.trim() || CAT[cat].name;
491     const exp = { id: uid(), date, cat, amount: Math.round(amount * 100) / 100, note };
492     state.expenses.push(exp);
493     persist() => api("POST", "expenses", toApi(exp));
494     month = date.slice(0, 7);
495     msg.className = "form-msg full";
496     msg.textContent = `Added ${money(amount)} to ${CAT[cat].name}.`;
497     $("fAmount").value = ""; $("fNote").value = "";
498     render();
499 });
500
501 let clearArmed = false;
502 $("clearBtn").addEventListener("click", () => {
503     const b = $("clearBtn");
504     if (!clearArmed) { clearArmed = true; b.textContent = "Click again to clear"; b.classList.add("danger-a
    ↪ rmed"); setTimeout(() => { clearArmed = false; b.textContent = "Clear sample data"; b.classList.remove("dan
    ↪ ger-armed"); }, 4000); return; }
505     state.expenses = []; state.sample = false; month = ym(today); render();
506     persist() => api("DELETE", "expenses");
507 });
508
509 $("search").addEventListener("input", e => { query = e.target.value.toLowerCase(); renderTable(); });
510
511 // ----- Charts -----
512 const charts = {};
513 function baseOpts() {
514     const ink2 = css("--ink-2"), grid = css("--grid"), surface = css("--surface"), ink = css("--ink"), line
    ↪ = css("--line");
515     return {
516         responsive: true, maintainAspectRatio: false, animation: { duration: 400 },
517         plugins: {
518             legend: { display: false },
519             tooltip: {
520                 backgroundColor: surface, titleColor: ink, bodyColor: ink2, borderColor: line, borderWidth: 1,
521                 padding: 10, cornerRadius: 8, boxPadding: 4, usePointStyle: true,
522                 titleFont: { family: "IBM Plex Sans", weight: "600" }, bodyFont: { family: "IBM Plex Mono" },
523             },
524         },
525         scales: {
526             x: { grid: { display: false }, border: { color: line }, ticks: { color: ink2, font: { family: "IBM
    ↪ Plex Sans", size: 12 } },
527             y: { beginAtZero: true, grid: { color: grid }, border: { display: false }, ticks: { color: ink2, fo
    ↪ nt: { family: "IBM Plex Mono", size: 11 }, callback: v => fmtShort.format(v), maxTicksLimit: 6 } },
528         },
529     };
530 }
531 function makeChart(id, cfg) { if (charts[id]) charts[id].destroy(); charts[id] = new Chart($(id), cfg); }
532
533 function renderTrend() {
534     const keys = Array.from({ length: 6 }, (_, i) => shiftMonth(month, i - 5));
535     const surface = css("--surface");
536     const datasets = CATS.map((c, i) => ({
537         label: c.name,
538         data: keys.map(k => byCat(inMonth(k))[c.id]),
539         backgroundColor: color(c.id),

```

```

540     borderColor: surface,
541     borderWidth: { top: 2, left: 0, right: 0, bottom: 0 },
542     borderRadius: i === CATS.length - 1 ? { topLeft: 4, topRight: 4 } : 0,
543     borderSkipped: "bottom",
544     maxBarThickness: 44,
545     stack: "s",
546   });
547   const opts = baseOpts();
548   opts.scales.x.stacked = true; opts.scales.y.stacked = true;
549   opts.interaction = { mode: "index", intersect: false };
550   opts.plugins.tooltip.itemSort = (a, b) => b.raw - a.raw;
551   opts.plugins.tooltip.filter = item => item.raw > 0;
552   opts.plugins.tooltip.callbacks = {
553     label: ctx => ` ${ctx.dataset.label}: ${money(ctx.raw)} `,
554     footer: items => `Total: ${money(items.reduce((a, i) => a + i.raw, 0))}`,
555   };
556   opts.plugins.tooltip.footerFont = { family: "IBM Plex Mono", weight: "600" };
557   opts.plugins.tooltip.footerColor = css("--ink");
558   makeChart("trendChart", { type: "bar", data: { labels: keys.map(k => monthDate(k).toLocaleDateString("en-US", { month: "short" }) + (k === month ? " ●" : "")), datasets }, options: opts });
559 }
560
561 function renderDonut(cats, total) {
562   const surface = css("--surface");
563   const order = CATS.filter(c => cats[c.id] > 0);
564   makeChart("donutChart", {
565     type: "doughnut",
566     data: { labels: order.map(c => c.name), datasets: [{ data: order.map(c => cats[c.id]), backgroundColor: order.map(c => color(c.id)), borderColor: surface, borderWidth: 2, hoverOffset: 6, borderRadius: 3 } ] },
567     options: {
568       responsive: true, maintainAspectRatio: false, cutout: "70%", layout: { padding: 6 },
569       plugins: {
570         legend: { display: false },
571         tooltip: { ...baseOpts().plugins.tooltip, callbacks: { label: ctx => ` ${money(ctx.raw)} · ${((ctx.x.raw / total) * 100).toFixed(1)}%` } },
572       },
573     },
574   });
575   $("donutTotal").textContent = money(total);
576   $("donutHint").textContent = monthLabel(month, "short");
577   const sorted = [...CATS].sort((a, b) => cats[b.id] - cats[a.id]);
578   $("catList").innerHTML = sorted.map(c => `<li><i class="swatch" style="--c:var(${c.v})"></i><span>${c.name}</span><span class="amt">${money(cats[c.id])}</span><span class="pct">${total ? Math.round(cats[c.id] / total * 100) : 0}%</span></li>`).join("");
579 }
580
581 function renderPace() {
582   const dim = daysIn(month);
583   const isCurrent = month === ym(today);
584   const lastDay = isCurrent ? today.getDate() : dim;
585   const cum = key => {
586     const n = daysIn(key); const daily = Array(n).fill(0);
587     inMonth(key).forEach(e => { daily[Number(e.date.slice(8, 10)) - 1] += e.amount; });
588     let run = 0; return daily.map(v => (run += v));
589   };
590   const cur = cum(month).map((v, i) => (i < lastDay ? v : null));
591   const prevRaw = cum(shiftMonth(month, -1));
592   const prev = Array.from({ length: dim }, (_, i) => prevRaw[Math.min(i, prevRaw.length - 1)]);
593   const tb = totalBudget();
594   const pace = Array.from({ length: dim }, (_, i) => Math.round(tb * (i + 1) / dim));
595   const c1 = css("--c1"), ghost = css("--ghost"), muted = css("--muted"), surface = css("--surface");
596   const opts = baseOpts();
597   opts.interaction = { mode: "index", intersect: false };
598   opts.scales.x.ticks.maxTicksLimit = 10;
599   opts.scales.x.ticks.autoSkip = true;
600   opts.plugins.tooltip.callbacks = {
601     title: items => monthDate(month).toLocaleDateString("en-US", { month: "short" }) + " " + items[0].label,
602     label: ctx => ctx.raw == null ? null : ` ${ctx.dataset.label}: ${money(ctx.raw)} `,
603   };
604   opts.plugins.tooltip.filter = item => item.raw != null;
605   makeChart("paceChart", {
606     type: "line",
607     data: {
608       labels: Array.from({ length: dim }, (_, i) => i + 1),
609       datasets: [
610         { label: "This month", data: cur, borderColor: c1, backgroundColor: c1 + "22", fill: "origin", borderWidth: 2, tension: 0.25, pointRadius: cur.map((v, i) => i === lastDay - 1 ? 5 : 0), pointBackgroundColor: c1, pointBorderColor: surface, pointBorderWidth: 2, pointHoverRadius: 5, order: 1 },

```

```

611     { label: "Previous month", data: prev, borderColor: ghost, borderWidth: 2, tension: 0.25, pointRadius: 0, pointHoverRadius: 4, fill: false, order: 2 },
612     { label: "Budget pace", data: pace, borderColor: muted, borderDash: [5, 4], borderWidth: 1.5, pointRadius: 0, pointHoverRadius: 0, fill: false, order: 3 },
613   ],
614   },
615   options: opts,
616 });
617 }
618
619 function renderDow(list) {
620   const names = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
621   const vals = Array(7).fill(0);
622   list.forEach(e => { const [y, m, d] = e.date.split("-").map(Number); vals[(new Date(y, m - 1, d).getDay() + 6) % 7] += e.amount; });
623   const opts = baseOpts();
624   opts.scales.y.ticks.maxTicksLimit = 4;
625   opts.plugins.tooltip.callbacks = { label: ctx => ` ${money(ctx.raw)} ` };
626   makeChart("dowChart", { type: "bar", data: { labels: names, datasets: [{ label: "Spent", data: vals, backgroundColor: css("--c1"), borderRadius: { topLeft: 4, topRight: 4 }, borderSkipped: "bottom", maxBarThickness: 28 }] }, options: opts });
627 }
628
629 // ----- KPI / budgets / table -----
630 function renderKpis(list, cats, total) {
631   const prevTotal = sum(inMonth(shiftMonth(month, -1)));
632   $("kSpent").textContent = money(total);
633   if (prevTotal > 0) {
634     const d = (total - prevTotal) / prevTotal;
635     const isCurrent = month === ym(today);
636     $("kSpentSub").innerHTML = `<span class="delta ${d > 0 ? "up" : "down"} num">${d > 0 ? "▲" : "▼"} ${Math.abs(d * 100).toFixed(0)}%</span> vs ${money(prevTotal)} ${isCurrent ? "all of" : "in"} ${monthDate(shiftMonth(month, -1)).toLocaleDateString("en-US", { month: "short" })}`;
637   } else $("kSpentSub").textContent = `${list.length} transactions`;
638
639   const tb = totalBudget(); const pct = tb ? total / tb : 0; const [cls, lbl] = status(pct);
640   $("kLeft").textContent = money(tb - total);
641   $("kMeter").style.width = Math.min(100, pct * 100).toFixed(1) + "%";
642   $("kMeter").style.background = `var(--${cls})`;
643   $("kLeftSub").innerHTML = `<span class="pill ${cls}">${lbl}</span><span class="num">${Math.round(pct * 100)}% of ${money(tb)}</span>`;
644
645   const isCurrent = month === ym(today);
646   const days = isCurrent ? today.getDate() : daysIn(month);
647   $("kDaily").textContent = money(days ? total / days : 0);
648   if (isCurrent) {
649     const proj = days ? total / days * daysIn(month) : 0;
650     $("kDailySub").innerHTML = `Projected month: <span class="num">${money(proj)}</span>`;
651   } else $("kDailySub").textContent = `Across ${days} days`;
652
653   const top = [...CATS].sort((a, b) => cats[b.id] - cats[a.id])[0];
654   if (total > 0) {
655     $("kTop").textContent = top.name;
656     $("kTopSub").innerHTML = `<i class="swatch" style="--c:var(${top.v})"></i><span class="num">${money(cats[top.id])}</span> · ${Math.round(cats[top.id] / total * 100)}% of spending`;
657   } else { $("kTop").textContent = "—"; $("kTopSub").textContent = "No spending yet"; }
658 }
659
660 function renderBudgets(cats) {
661   $("budgetList").innerHTML = CATS.map(c => {
662     const b = state.budgets[c.id] || 0, s = cats[c.id]; const pct = b ? s / b : (s ? 2 : 0); const [cls, lbl] = status(pct);
663     return `<li>
664       <div class="row">
665         <span class="name"><i class="swatch" style="--c:var(${c.v})"></i>${c.name} <span class="pill ${cls}">${lbl}</span></span>
666         <span class="fig">${money(s)} /
667         <input type="number" min="0" step="100" id="bud-${c.id}" data-cat="${c.id}" value="${b}" aria-label="${c.name} monthly budget"></span>
668       </div>
669       <div class="bar${pct > 1 ? " over" : ""}" style="--c:var(${c.v})" role="img" aria-label="${c.name}: ${Math.round(pct * 100)}% of budget used"><span style="width:${Math.min(100, pct * 100).toFixed(1)}%></span></div>
670     </li>`;
671   }).join("");
672   $("budgetList").querySelectorAll("input").forEach(inp => inp.addEventListener("change", e => {
673     const v = Math.max(0, parseFloat(e.target.value) || 0);
674     const cat = e.target.dataset.cat;
675     state.budgets[cat] = v; render();
676   }));

```

```

676     persist(() => api("PUT", `budgets/${cat}`, { amount: v }));
677   });
678 }
679
680 function renderFilters() {
681   const chips = [{ id: "all", name: "All" }, ...CATS];
682   $("filters").innerHTML = chips.map(c => `<button type="button" class="chip" data-cat="${c.id}" aria-pre
    ssed="${filterCat === c.id}">${c.v} ? `<i class="swatch" style="--c:var(${c.v})"></i>` : ""}${c.name}</butto
    n>`).join("");
683   $("filters").querySelectorAll(".chip").forEach(b => b.addEventListener("click", () => { filterCat = b.d
    ataset.cat; renderFilters(); renderTable(); }));
684 }
685
686 function renderTable() {
687   let list = inMonth(month).filter(e => (filterCat === "all" || e.cat === filterCat) && (!query || e.note
    .toLowerCase().includes(query)));
688   list.sort((a, b) => b.date.localeCompare(a.date) || b.amount - a.amount);
689   $("tbody").innerHTML = list.length ? list.map(e => {
690     const [y, m, d] = e.date.split("-").map(Number);
691     const ds = new Date(y, m - 1, d).toLocaleDateString("en-US", { weekday: "short", month: "short", day:
    "numeric" });
692     return `<tr><td>${ds}</td><td><span class="cat-tag"><i class="swatch" style="--c:var(${CAT[e.cat].v})
    "></i>${CAT[e.cat].name}</span></td><td class="note">${esc(e.note)}</td><td class="amt">${money(e.amount)}<
    /td><td><button class="del" data-id="${e.id}" aria-label="Delete ${esc(e.note)}" title="Delete"><</button><
    /td></tr>`;
693   }).join("") : `<tr><td colspan="5" class="empty">No expenses match. Add one with the form, or change th
    e filter.</td></tr>`;
694   $("tbody").querySelectorAll(".del").forEach(b => b.addEventListener("click", () => {
695     const id = b.dataset.id;
696     state.expenses = state.expenses.filter(e => e.id !== id); render();
697     persist(() => api("DELETE", `expenses/${encodeURIComponent(id)}`));
698   }));
699   $("txCount").textContent = `${list.length} transaction${list.length === 1 ? "" : "s"}`;
700   $("txSum").textContent = `Total ${money(sum(list))}`;
701 }
702
703 function render() {
704   fillMonths();
705   $("sampleNotice").hidden = !state.sample;
706   const list = inMonth(month), cats = byCat(list), total = sum(list);
707   renderKpis(list, cats, total);
708   renderTrend();
709   renderDonut(cats, total);
710   renderPace();
711   renderBudgets(cats);
712   renderDow(list);
713   renderFilters();
714   renderTable();
715 }
716
717 async function start() {
718   if (await detectServer()) {
719     try {
720       await loadFromServer();
721       serverMode = true;
722       $("storeLabel").textContent = "Saved on your server";
723       $("storeLabel").classList.add("server");
724       $("storeText").textContent = "Changes are saved to your server.";
725       $("csvLink").hidden = false;
726       $("curSel").value = state.currency;
727     } catch (err) {
728       showSyncError(`Couldn't load your data from the server (${err.message}). Showing this browser's cop
    y instead; changes won't reach the server until you reload.`);
729     }
730   }
731   setCurrency();
732   if (window.Chart) { Chart.defaults.font.family = "IBM Plex Sans, system-ui, sans-serif"; }
733   render();
734 }
735
736 // Re-render charts when the theme changes so they pick up the new tokens.
737 const rerender = () => { if (window.Chart) render(); };
738 try { matchMedia("(prefers-color-scheme: dark)").addEventListener("change", rerender); } catch (e) {}
739 new MutationObserver(rerender).observe(document.documentElement, { attributes: true, attributeFilter: ["d
    ata-theme"] });
740
741 if (document.fonts && document.fonts.ready) document.fonts.ready.then(() => { if (charts.trendChart) rend
    er(); });
742 start();

```

```
743 }>();  
744 </script>  
745 <!-- ARTIFACT-BODY-END -->  
746 </body>  
747 </html>
```

backend/package.json: Dependencies and npm scripts

```
1 {
2   "name": "pocket-ledger-backend",
3   "version": "1.0.0",
4   "description": "REST API and SQLite storage for the Pocket Ledger expense dashboard",
5   "private": true,
6   "type": "module",
7   "main": "src/server.js",
8   "engines": {
9     "node": ">=22.13"
10  },
11  "scripts": {
12    "start": "node --disable-warning=ExperimentalWarning src/server.js",
13    "dev": "node --disable-warning=ExperimentalWarning --watch src/server.js",
14    "test": "node --disable-warning=ExperimentalWarning --test test/*.test.js"
15  },
16  "dependencies": {
17    "express": "^5.2.1"
18  }
19 }
```

backend/src/categories.js: Category ids, default budgets, currencies

```
1 // Expense categories. Ids must match the CATS list in ../index.html.
2 export const CATEGORIES = [
3   { id: "housing",    name: "Housing",      budget: 18000 },
4   { id: "food",       name: "Food & Dining", budget: 12000 },
5   { id: "transport",  name: "Transport",     budget: 6000 },
6   { id: "utilities",  name: "Utilities",     budget: 4500 },
7   { id: "shopping",   name: "Shopping",      budget: 6000 },
8   { id: "health",     name: "Health",        budget: 3000 },
9   { id: "fun",        name: "Entertainment", budget: 3500 },
10  { id: "other",      name: "Other",          budget: 2500 },
11 ];
12
13 export const CATEGORY_IDS = new Set(CATEGORIES.map(c => c.id));
14
15 export const CURRENCIES = new Set(["INR", "USD", "EUR", "GBP"]);
```

backend/src/db.js: SQLite schema and data access

```
1 import { DatabaseSync } from "node:sqlite";
2 import { mkdirSync } from "node:fs";
3 import { dirname } from "node:path";
4 import { CATEGORIES } from "../categories.js";
5
6 const SCHEMA = `
7   CREATE TABLE IF NOT EXISTS expenses (
8     id          TEXT PRIMARY KEY,
9     date        TEXT NOT NULL,
10    category     TEXT NOT NULL,
11    amount       REAL NOT NULL CHECK (amount > 0),
12    note         TEXT NOT NULL DEFAULT '',
13    created_at   TEXT NOT NULL DEFAULT (datetime('now'))
14  );
15  CREATE INDEX IF NOT EXISTS idx_expenses_date ON expenses (date);
16
17  CREATE TABLE IF NOT EXISTS budgets (
18    category TEXT PRIMARY KEY,
19    amount   REAL NOT NULL CHECK (amount >= 0)
20  );
21
22  CREATE TABLE IF NOT EXISTS settings (
23    key   TEXT PRIMARY KEY,
24    value TEXT NOT NULL
25  );
26 `;
27
28 // Opens (or creates) the database and returns a small data-access layer.
29 // Pass ":memory:" for a throwaway database (used by the tests).
30 export function openDb(path) {
31   if (path !== ":memory:") mkdirSync(dirname(path), { recursive: true });
32   const db = new DatabaseSync(path);
33   db.exec("PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON;");
34   db.exec(SCHEMA);
35
36   // SIP and EMI were retired as categories: drop them from sample data, move the user's own entries to 0th
37   // er.
38   const isSample = db.prepare("SELECT value FROM settings WHERE key = 'sample'").get()?.value === "true";
39   db.prepare(isSample
40     ? "DELETE FROM expenses WHERE category IN ('sip', 'emi')"
41     : "UPDATE expenses SET category = 'other' WHERE category IN ('sip', 'emi')").run();
42   db.prepare("DELETE FROM budgets WHERE category IN ('sip', 'emi')").run();
43
44   const insertBudget = db.prepare("INSERT OR IGNORE INTO budgets (category, amount) VALUES (?, ?)");
45   for (const c of CATEGORIES) insertBudget.run(c.id, c.budget);
46
47   const q = {
48     allExpenses: db.prepare("SELECT id, date, category, amount, note FROM expenses ORDER BY date DESC, amou
49     nt DESC"),
50     monthExpenses: db.prepare("SELECT id, date, category, amount, note FROM expenses WHERE substr(date, 1,
51     7) = ? ORDER BY date DESC, amount DESC"),
52     getExpense: db.prepare("SELECT id, date, category, amount, note FROM expenses WHERE id = ?"),
53     insertExpense: db.prepare("INSERT INTO expenses (id, date, category, amount, note) VALUES (?, ?, ?, ?,
54     ?)"),
55     updateExpense: db.prepare("UPDATE expenses SET date = ?, category = ?, amount = ?, note = ? WHERE id =
56     ?"),
57     deleteExpense: db.prepare("DELETE FROM expenses WHERE id = ?"),
58     deleteAll: db.prepare("DELETE FROM expenses"),
59     budgets: db.prepare("SELECT category, amount FROM budgets"),
60     setBudget: db.prepare("INSERT INTO budgets (category, amount) VALUES (?, ?) ON CONFLICT (category) DO U
61     PDATE SET amount = excluded.amount"),
62     settings: db.prepare("SELECT key, value FROM settings"),
63     setSetting: db.prepare("INSERT INTO settings (key, value) VALUES (?, ?) ON CONFLICT (key) DO UPDATE SET
64     value = excluded.value"),
65     summary: db.prepare("SELECT category, SUM(amount) AS total, COUNT(*) AS count FROM expenses WHERE subst
66     r(date, 1, 7) = ? GROUP BY category"),
67   };
68
69   const tx = fn => {
70     db.exec("BEGIN");
71     try { const r = fn(); db.exec("COMMIT"); return r; }
72     catch (e) { db.exec("ROLLBACK"); throw e; }
73   };
74
75   return {
76     listExpenses: month => (month ? q.monthExpenses.all(month) : q.allExpenses.all()).map(plain),
77     getExpense: id => plain(q.getExpense.get(id)),
78   };
79 }
```

```

70     addExpense: e => { q.insertExpense.run(e.id, e.date, e.category, e.amount, e.note); return plain(q.getExpense.get(e.id)); },
71     updateExpense: (id, e) => q.updateExpense.run(e.date, e.category, e.amount, e.note, id).changes > 0,
72     deleteExpense: id => q.deleteExpense.run(id).changes > 0,
73     clearExpenses: () => tx(() => { q.deleteAll.run(); q.setSetting.run("sample", "false"); }),
74
75     budgets: () => Object.fromEntries(q.budgets.all().map(r => [r.category, r.amount])),
76     setBudget: (category, amount) => q.setBudget.run(category, amount),
77
78     settings: () => Object.fromEntries(q.settings.all().map(r => [r.key, r.value])),
79     setSetting: (key, value) => q.setSetting.run(key, String(value)),
80
81     summary: month => q.summary.all(month).map(plain),
82
83     // Loads a full ledger in one transaction (first-run sample data, or data moved from browser storage).
84     importLedger: ({ expenses, budgets, currency, sample, replace }) => tx(() => {
85         if (replace) q.deleteAll.run();
86         for (const e of expenses) q.insertExpense.run(e.id, e.date, e.category, e.amount, e.note);
87         for (const [c, v] of Object.entries(budgets || {})) q.setBudget.run(c, v);
88         if (currency) q.setSetting.run("currency", currency);
89         q.setSetting.run("sample", String(Boolean(sample)));
90         q.setSetting.run("initialized", "true");
91     }),
92
93     close: () => db.close(),
94 };
95 }
96
97 // node:sqlite returns null-prototype rows; convert them to ordinary objects for JSON.
98 const plain = row => (row ? { ...row } : row);

```

backend/src/server.js: Express REST API, validation, login

```
1 import express from "express";
2 import { randomUUID, timingSafeEqual } from "node:crypto";
3 import { fileURLToPath } from "node:url";
4 import { dirname, join, resolve } from "node:path";
5 import { openDb } from "../db.js";
6 import { CATEGORIES, CATEGORY_IDS, CURRENCIES } from "../categories.js";
7
8 const HERE = dirname(fileURLToPath(import.meta.url));
9 const FRONTEND = resolve(HERE, "../../index.html");
10 const MAX_AMOUNT = 1e9;
11
12 class HttpError extends Error {
13   constructor(status, message) { super(message); this.status = status; }
14 }
15
16 // ----- Validation -----
17 function isRealDate(s) {
18   if (typeof s !== "string" || !/^\d{4}-\d{2}-\d{2}$/.test(s)) return false;
19   const [y, m, d] = s.split("-").map(Number);
20   const dt = new Date(Date.UTC(y, m - 1, d));
21   return dt.getUTCFullYear() === y && dt.getUTCMonth() === m - 1 && dt.getUTCDate() === d;
22 }
23
24 function parseExpense(body, { requireId = false } = {}) {
25   if (!body || typeof body !== "object") throw new HttpError(400, "Send the expense as a JSON object.");
26   const { id, date, category, amount, note = "" } = body;
27   if (!isRealDate(date)) throw new HttpError(400, "date must be a real calendar date in YYYY-MM-DD format.");
28   if (!CATEGORY_IDS.has(category)) throw new HttpError(400, `category must be one of: ${[...CATEGORY_IDS].join(", ")}`);
29   if (typeof amount !== "number" || !Number.isFinite(amount) || amount <= 0 || amount > MAX_AMOUNT)
30     throw new HttpError(400, "amount must be a number greater than 0.");
31   if (typeof note !== "string" || note.length > 200) throw new HttpError(400, "note must be text of at most 200 characters.");
32   if (id !== undefined && (typeof id !== "string" || !/^[A-Za-z0-9_-]{4,64}$/.test(id)))
33     throw new HttpError(400, "id may only contain letters, digits, '-' and '_' (4 to 64 characters).");
34   if (requireId && id === undefined) throw new HttpError(400, "id is required.");
35   return { id: id ?? randomUUID(), date, category, amount: Math.round(amount * 100) / 100, note: note.trim() };
36 }
37
38 function parseBudget(amount) {
39   if (typeof amount !== "number" || !Number.isFinite(amount) || amount < 0 || amount > MAX_AMOUNT)
40     throw new HttpError(400, "amount must be a number of 0 or more.");
41   return Math.round(amount * 100) / 100;
42 }
43
44 function parseMonth(month) {
45   if (month === undefined) return undefined;
46   if (typeof month !== "string" || !/^\d{4}-(0[1-9]|1[0-2])$/.test(month)) throw new HttpError(400, "month must be in YYYY-MM format.");
47   return month;
48 }
49
50 const csvCell = v => {
51   let s = String(v ?? "");
52   if (/^[=+\-@\t\r]$/i.test(s)) s = "'" + s; // stop spreadsheets from running cell text as a formula
53   return /[",\n\r]$/i.test(s) ? `"${s.replace(/"/g, '"')}"` : s;
54 };
55
56 // Optional HTTP Basic auth: set APP_PASSWORD (and optionally APP_USER) to require a login.
57 function basicAuth(user, password) {
58   const expected = Buffer.from(`${user}:${password}`);
59   return (req, res, next) => {
60     const header = req.get("authorization") || "";
61     const given = Buffer.from(header.startsWith("Basic ") ? Buffer.from(header.slice(6), "base64").toString() : "");
62     if (given.length === expected.length && timingSafeEqual(given, expected)) return next();
63     res.set("WWW-Authenticate", `Basic realm="Pocket Ledger", charset="UTF-8"`).status(401).send("Login required.");
64   };
65 }
66
67 // ----- App -----
68 export function createApp(db, { password = process.env.APP_PASSWORD, user = process.env.APP_USER || "ledger" } = {}) {
69   const app = express();
```

```

70 app.disable("x-powered-by");
71 app.use((req, res, next) => { res.set("X-Content-Type-Options", "nosniff"); next(); });
72 if (password) app.use(basicAuth(user, password));
73 app.use(express.json({ limit: "5mb" }));
74
75 const api = express.Router();
76
77 api.get("/health", (req, res) => res.json({ ok: true }));
78
79 api.get("/categories", (req, res) => res.json(CATEGORIES));
80
81 // Everything the dashboard needs in one request.
82 api.get("/state", (req, res) => {
83   const s = db.settings();
84   res.json({
85     initialized: s.initialized === "true",
86     sample: s.sample === "true",
87     currency: s.currency || "INR",
88     budgets: db.budgets(),
89     expenses: db.listExpenses(),
90   });
91 });
92
93 api.get("/expenses", (req, res) => {
94   const month = parseMonth(req.query.month);
95   const category = req.query.category;
96   if (category !== undefined && !CATEGORY_IDS.has(category)) throw new HttpError(400, "Unknown category."
    );
97   let rows = db.listExpenses(month);
98   if (category) rows = rows.filter(r => r.category === category);
99   res.json(rows);
100 });
101
102 api.get("/expenses/:id", (req, res) => {
103   const row = db.getExpense(req.params.id);
104   if (!row) throw new HttpError(404, "No expense with that id.");
105   res.json(row);
106 });
107
108 api.post("/expenses", (req, res) => {
109   const e = parseExpense(req.body);
110   if (db.getExpense(e.id)) throw new HttpError(409, "An expense with that id already exists.");
111   res.status(201).json(db.addExpense(e));
112 });
113
114 api.put("/expenses/:id", (req, res) => {
115   const e = parseExpense({ ...req.body, id: req.params.id });
116   if (!db.updateExpense(req.params.id, e)) throw new HttpError(404, "No expense with that id.");
117   res.json(db.getExpense(req.params.id));
118 });
119
120 api.delete("/expenses/:id", (req, res) => {
121   if (!db.deleteExpense(req.params.id)) throw new HttpError(404, "No expense with that id.");
122   res.status(204).end();
123 });
124
125 // Removes every expense (used by "Clear sample data"). Budgets and settings are kept.
126 api.delete("/expenses", (req, res) => {
127   db.clearExpenses();
128   res.status(204).end();
129 });
130
131 api.get("/budgets", (req, res) => res.json(db.budgets()));
132
133 api.put("/budgets/:category", (req, res) => {
134   if (!CATEGORY_IDS.has(req.params.category)) throw new HttpError(404, "Unknown category.");
135   db.setBudget(req.params.category, parseBudget(req.body?.amount));
136   res.json(db.budgets());
137 });
138
139 api.put("/settings", (req, res) => {
140   const { currency } = req.body || {};
141   if (!CURRENCIES.has(currency)) throw new HttpError(400, `currency must be one of: ${[...CURRENCIES].join(
    ", "
  )}`);
142   db.setSetting("currency", currency);
143   res.json({ currency });
144 });
145
146 // Month totals per category, for other clients or reports.

```

```

147 api.get("/summary", (req, res) => {
148   const month = parseMonth(req.query.month);
149   if (!month) throw new HttpError(400, "month is required, in YYYY-MM format.");
150   const rows = db.summary(month);
151   const budgets = db.budgets();
152   const total = rows.reduce((a, r) => a + r.total, 0);
153   const budgetTotal = Object.values(budgets).reduce((a, v) => a + v, 0);
154   res.json({
155     month, total, budgetTotal, remaining: budgetTotal - total,
156     categories: CATEGORIES.map(c => {
157       const r = rows.find(x => x.category === c.id);
158       return { id: c.id, name: c.name, spent: r?.total ?? 0, count: r?.count ?? 0, budget: budgets[c.id]
159         ?? 0 };
160     }),
161   });
162
163   // Loads a whole ledger at once: first-run sample data, or data moved over from browser storage.
164   // Refuses to overwrite an existing ledger unless "replace": true is sent.
165   api.post("/import", (req, res) => {
166     const { expenses, budgets = {}, currency, sample = false, replace = false } = req.body || {};
167     if (!Array.isArray(expenses) || expenses.length > 50000) throw new HttpError(400, "expenses must be an
168       array of at most 50,000 items.");
169     if (db.settings().initialized === "true" && !replace) throw new HttpError(409, "The ledger already has
170       data. Send \"replace\": true to overwrite it.");
171     const parsed = expenses.map(e => parseExpense(e));
172     if (new Set(parsed.map(e => e.id)).size !== parsed.length) throw new HttpError(400, "Expense ids must be
173       e unique.");
174     const cleanBudgets = {};
175     for (const [c, v] of Object.entries(budgets)) {
176       if (!CATEGORY_IDS.has(c)) throw new HttpError(400, `Unknown budget category: ${c}.`);
177       cleanBudgets[c] = parseBudget(v);
178     }
179     if (currency !== undefined && !CURRENCIES.has(currency)) throw new HttpError(400, "Unknown currency.");
180     db.importLedger({ expenses: parsed, budgets: cleanBudgets, currency, sample, replace });
181     res.status(201).json({ imported: parsed.length });
182   });
183
184   // Spreadsheet-friendly download of every expense.
185   api.get("/export.csv", (req, res) => {
186     const rows = db.listExpenses();
187     const name = Object.fromEntries(CATEGORIES.map(c => [c.id, c.name]));
188     const lines = ["date,category,amount,note", ...rows.map(r => [r.date, name[r.category] ?? r.category, r
189       .amount, r.note].map(csvCell).join(","))];
190     res.set("Content-Type", "text/csv; charset=utf-8");
191     res.set("Content-Disposition", "attachment; filename=pocket-ledger.csv");
192     res.send(lines.join("\n") + "\n");
193   });
194
195   app.use("/api", api);
196   app.use("/api", (req, res) => res.status(404).json({ error: "No such API endpoint." }));
197
198   // The dashboard itself.
199   app.get(["/", "/index.html"], (req, res) => res.sendFile(FRONTEND));
200
201   app.use((err, req, res, next) => {
202     const status = err.status || err.statusCode || 500;
203     if (status >= 500) console.error(err);
204     const message = err instanceof HttpError ? err.message
205       : err.type === "entity.parse.failed" ? "Request body is not valid JSON."
206       : err.type === "entity.too.large" ? "Request body is too large."
207       : "Something went wrong on the server.";
208     res.status(status).json({ error: message });
209   });
210
211   return app;
212 }
213
214 // ----- Start -----
215 if (process.argv[1] && resolve(process.argv[1]) === fileURLToPath(import.meta.url)) {
216   const port = Number(process.env.PORT) || 3000;
217   const dbPath = process.env.DB_PATH || join(HERE, "../data/ledger.db");
218   const db = openDb(dbPath);
219   const server = createApp(db).listen(port, () => {
220     console.log(`Pocket Ledger running at http://localhost:${port} (database: ${dbPath})`);
221     if (!process.env.APP_PASSWORD) console.log("Tip: set APP_PASSWORD to require a login before exposing th
222       is to the internet.");
223   });
224   const stop = () => server.close(() => { db.close(); process.exit(0); });

```

```
220 process.on("SIGINT", stop);
221 process.on("SIGTERM", stop);
222 }
```

backend/test/api.test.js: Automated API tests

```
1 import { test, before, after } from "node:test";
2 import assert from "node:assert/strict";
3 import { openDb } from "../src/db.js";
4 import { createApp } from "../src/server.js";
5
6 let server, base, db;
7
8 before(async () => {
9   db = openDb(":memory:");
10  server = createApp(db, { password: "" }).listen(0);
11  await new Promise(r => server.once("listening", r));
12  base = `http://127.0.0.1:${server.address().port}`;
13 });
14 after(() => { server.close(); db.close(); });
15
16 const call = async (method, path, body) => {
17   const res = await fetch(base + path, {
18     method,
19     headers: body !== undefined ? { "Content-Type": "application/json" } : {},
20     body: body !== undefined ? JSON.stringify(body) : undefined,
21   });
22   const text = await res.text();
23   let json; try { json = JSON.parse(text); } catch { json = text; }
24   return { status: res.status, body: json, headers: res.headers };
25 };
26
27 test("health check and dashboard page are served", async () => {
28   assert.deepEqual(await call("GET", "/api/health").body, { ok: true });
29   const page = await fetch(base + "/");
30   assert.equal(page.status, 200);
31   assert.match(await page.text(), /Pocket Ledger/);
32 });
33
34 test("a fresh database is uninitialized with default budgets", async () => {
35   const { body } = await call("GET", "/api/state");
36   assert.equal(body.initialized, false);
37   assert.equal(body.currency, "INR");
38   assert.equal(body.budgets.housing, 18000);
39   assert.equal(body.budgets.other, 2500);
40   assert.equal(body.budgets.emi, undefined);
41   assert.deepEqual(body.expenses, []);
42 });
43
44 test("import loads a ledger once and refuses to overwrite without replace", async () => {
45   const ledger = {
46     expenses: [
47       { id: "seed0001", date: "2026-09-01", category: "housing", amount: 18000, note: "Rent" },
48       { id: "seed0002", date: "2026-09-05", category: "health", amount: 9800, note: "Hospital bill" },
49       { id: "seed0003", date: "2026-08-07", category: "shopping", amount: 5000, note: "New phone" },
50     ],
51     budgets: { food: 10000 },
52     currency: "INR",
53     sample: true,
54   };
55   assert.equal(await call("POST", "/api/import", ledger).status, 201);
56   assert.equal(await call("POST", "/api/import", ledger).status, 409);
57
58   const { body } = await call("GET", "/api/state");
59   assert.equal(body.initialized, true);
60   assert.equal(body.sample, true);
61   assert.equal(body.expenses.length, 3);
62   assert.equal(body.budgets.food, 10000);
63 });
64
65 test("expenses can be added, read, updated, filtered and deleted", async () => {
66   const created = await call("POST", "/api/expenses", { id: "abc12345", date: "2026-09-10", category: "food",
67     ↪ amount: 450.456, note: "Lunch" });
68   assert.equal(created.status, 201);
69   assert.deepEqual(created.body, { id: "abc12345", date: "2026-09-10", category: "food", amount: 450.46, no
70     ↪ te: "Lunch" });
71
72   assert.equal(await call("POST", "/api/expenses", { id: "abc12345", date: "2026-09-10", category: "food",
73     ↪ amount: 1 })).status, 409);
74
75   const noId = await call("POST", "/api/expenses", { date: "2026-09-11", category: "transport", amount: 120
76     ↪ });
77   assert.equal(noId.status, 201);
78 }
```

```

74   assert.ok(noId.body.id.length >= 4);
75
76   const updated = await call("PUT", "/api/expenses/abc12345", { date: "2026-09-12", category: "food", amount: 500, note: "Team lunch" });
77   assert.equal(updated.body.amount, 500);
78   assert.equal(updated.body.note, "Team lunch");
79
80   const sept = await call("GET", "/api/expenses?month=2026-09&category=food");
81   assert.deepEqual(sept.body.map(e => e.id), ["abc12345"]);
82
83   assert.equal((await call("DELETE", "/api/expenses/abc12345")).status, 204);
84   assert.equal((await call("GET", "/api/expenses/abc12345")).status, 404);
85   assert.equal((await call("DELETE", "/api/expenses/abc12345")).status, 404);
86 });
87
88 test("invalid input is rejected with a helpful message", async () => {
89   const cases = [
90     { date: "2026-02-30", category: "food", amount: 10 },
91     { date: "2026-09-10", category: "crypto", amount: 10 },
92     { date: "2026-09-10", category: "sip", amount: 10 },
93     { date: "2026-09-10", category: "food", amount: -5 },
94     { date: "2026-09-10", category: "food", amount: "10" },
95     { date: "2026-09-10", category: "food", amount: 10, id: "a b" },
96     { date: "2026-09-10", category: "food", amount: 10, note: "x".repeat(201) },
97   ];
98   for (const c of cases) {
99     const r = await call("POST", "/api/expenses", c);
100    assert.equal(r.status, 400, JSON.stringify(c));
101    assert.equal(typeof r.body.error, "string");
102   }
103   const bad = await fetch(base + "/api/expenses", { method: "POST", headers: { "Content-Type": "application/json" }, body: "{oops}" });
104   assert.equal(bad.status, 400);
105   assert.equal((await call("GET", "/api/expenses?month=2026-13")).status, 400);
106   assert.equal((await call("GET", "/api/nope")).status, 404);
107 });
108
109 test("budgets and currency can be changed", async () => {
110   const b = await call("PUT", "/api/budgets/health", { amount: 15000 });
111   assert.equal(b.body.health, 15000);
112   assert.equal((await call("PUT", "/api/budgets/health", { amount: -1 })).status, 400);
113   assert.equal((await call("PUT", "/api/budgets/emi", { amount: 1 })).status, 404);
114   assert.equal((await call("PUT", "/api/budgets/nothing", { amount: 1 })).status, 404);
115
116   assert.deepEqual((await call("PUT", "/api/settings", { currency: "USD" })).body, { currency: "USD" });
117   assert.equal((await call("PUT", "/api/settings", { currency: "BTC" })).status, 400);
118   assert.equal((await call("GET", "/api/state")).body.currency, "USD");
119 });
120
121 test("summary totals a month by category", async () => {
122   const { body } = await call("GET", "/api/summary?month=2026-09");
123   assert.equal(body.month, "2026-09");
124   const health = body.categories.find(c => c.id === "health");
125   assert.equal(health.spent, 9800);
126   assert.equal(health.budget, 15000);
127   assert.equal(body.total, body.categories.reduce((a, c) => a + c.spent, 0));
128   assert.equal((await call("GET", "/api/summary")).status, 400);
129 });
130
131 test("CSV export escapes text and neutralises formulas", async () => {
132   await call("POST", "/api/expenses", { id: "csv00001", date: "2026-09-20", category: "other", amount: 10, note: "HYPERLINK(\"x\")", "quoted" });
133   const res = await fetch(base + "/api/export.csv");
134   assert.match(res.headers.get("content-type"), /text\/csv/);
135   const text = await res.text();
136   assert.match(text, /^date,category,amount,note\n/);
137   assert.ok(text.includes(`2026-09-20,other,10,"'=HYPERLINK("x")", "quoted"`));
138 });
139
140 test("clearing removes all expenses and turns off the sample flag", async () => {
141   assert.equal((await call("DELETE", "/api/expenses")).status, 204);
142   const { body } = await call("GET", "/api/state");
143   assert.deepEqual(body.expenses, []);
144   assert.equal(body.sample, false);
145   assert.equal(body.budgets.health, 15000);
146 });
147
148 test("APP_PASSWORD turns on a login", async () => {
149   const locked = createApp(openDb(":memory:"), { user: "me", password: "s3cret" }).listen(0);

```

```

150   await new Promise(r => locked.once("listening", r));
151   const url = `http://127.0.0.1:${locked.address().port}/api/health`;
152   try {
153     assert.equal((await fetch(url)).status, 401);
154     const wrong = await fetch(url, { headers: { Authorization: "Basic " + Buffer.from("me:nope").toString("
↳ base64") } });
155     assert.equal(wrong.status, 401);
156     const ok = await fetch(url, { headers: { Authorization: "Basic " + Buffer.from("me:s3cret").toString("b
↳ ase64") } });
157     assert.equal(ok.status, 200);
158   } finally { locked.close(); }
159 });

```

Dockerfile: Container image for deployment

```
1 FROM node:22-alpine
2 WORKDIR /app
3 COPY backend/package.json backend/package-lock.json ./backend/
4 RUN cd backend && npm ci --omit=dev
5 COPY index.html ./
6 COPY backend/src ./backend/src
7 ENV NODE_ENV=production PORT=3000 DB_PATH=/data/ledger.db
8 RUN mkdir -p /data && chown node:node /data
9 VOLUME /data
10 EXPOSE 3000
11 USER node
12 CMD ["node", "--disable-warning=ExperimentalWarning", "backend/src/server.js"]
```